
Übungsblatt 1

Abgabe: 27.04.2026, 10:00 Uhr (im Digicampus via VIPS: .java-Dateien für Code, .uxf für UML, .pdf für alles andere)

- Dieses Übungsblatt muss im Team abgegeben werden (Einzelabgaben sind nicht erlaubt!).
- Die **Zeitangaben** geben zur Orientierung an, wie viel Zeit für eine Aufgabe später in der Klausur vorgesehen wäre; gehen Sie davon aus, dass Sie zum jetzigen Zeitpunkt wesentlich länger brauchen und die angegebene Zeit erst nach ausreichender Übung erreichen.

* leichte Aufgabe / ** mittelschwere Aufgabe / *** schwere Aufgabe

Allgemeine Hinweise zum Übungsbetrieb

- In den Tutorien werden Sie in Teams von jeweils 3 Personen eingeteilt, wenn Sie nicht schon ein Team haben. Dies dient dem Zweck, dass Sie schon jetzt Erfahrung damit sammeln, mit anderen Personen gemeinsam produktiv zu arbeiten. Ihr gebildetes Team hinterlegen Sie bitte im Digicampus, indem Sie in der Veranstaltung „Informatik 2“ auf den Tab **Vips** und anschließend links auf **Übungsgruppen** klicken. Suchen Sie sich danach in der angezeigten Liste eine Gruppe aus, in der noch genügend Platz für alle Team-Mitglieder ist und klicken Sie rechts von dem Gruppennamen auf das Symbol, um sich zur Gruppe hinzuzufügen. Dies müssen **alle** Teammitglieder machen.

Im Idealfall teilen Sie sich die zu erledigenden Aufgaben eines Übungsblattes **nicht** im Team auf, sondern bearbeiten unabhängig voneinander alle Aufgaben vom Übungsblatt. Anschließend diskutieren Sie im Team über Ihre Lösungen und einigen sich auf **eine** gemeinsame Lösung, die Sie abgeben möchten.

Ihre Lösung zu einem Übungsblatt muss nur von einer Person aus Ihrem Team abgegeben werden. Eine Anleitung dazu, wie Sie dies machen, finden Sie im Digicampus oder unter <https://mediastore.rz.uni-augsburg.de/get/aphEoyIlhN/>.

- Neben den Tutorien bieten wir jede Woche den *Offenen Inforaum* an: Dieser findet jeden **Mittwoch** von **14:00 bis 17:15 Uhr** im Raum **1002 N** statt. Es ist eine zusätzliche Möglichkeit, um sich mit Ihrem Team zu treffen und weiter am Übungsblatt zu arbeiten. Für Fragen steht Ihnen ein Mitarbeiter zur Verfügung. Sie können im Offenen Inforaum auch Fragen stellen, die sich nicht auf das aktuelle Übungsblatt beziehen. Der erste Offene Inforaum findet am **Mittwoch, den 22.04.26** statt.

Allgemeine Hinweise zur Abgabe der Übungsblätter

Wir geben Ihnen die Möglichkeit, Ihre Lösung der wöchentlich herausgegebenen Übungsblätter abzugeben und von einem Tutor korrigieren zu lassen. So erhalten Sie zeitnah Feedback, wo Sie stehen und erhalten im Idealfall noch einen Notenbonus auf eine bestandene Klausur. Folgende Punkte sind dabei zu beachten:

- **Abgabeort:** Die Abgabe erfolgt digital als Upload im Vips-Tool von Digicampus. Eine Anleitung dafür, wie man seine Lösung für ein Übungsblatt in Vips hochlädt, finden Sie im Digicampus oder unter <https://mediastore.rz.uni-augsburg.de/get/aphEoyI1hN/>.
- **Abgabeformat:** Für Programmieraufgaben sind alle Quellcode-Dateien als *.java-Dateien abzugeben. Diese Dateien legen Sie in einen Ordner mit dem Namen `src`, sodass der Tutor die Dateien leicht finden und ausführen kann. Neben diesem `src`-Ordner gehört **eine einzelne *.pdf-Datei** zu Ihrer Abgabe, in der Sie den Code für jede Abgabe lesbar einbinden, damit der Tutor seine Kommentare zum Code direkt in die PDF-Datei eintragen kann. In dieser *.pdf-Datei geben Sie auch alle weiteren Aufgaben ab (z.B. als gut lesbaren Scan Ihrer handschriftlichen Bearbeitung). Den `src`-Ordner und die *.pdf-Datei verpacken Sie zusammen in ein *.zip-Archiv, das Sie in Vips hochladen. Aus dem Namen des *.zip-Archivs sollte hervorgehen, zu welchem Team die Abgabe gehört (zum Beispiel durch Verwendung der in Vips gewählten Gruppennummer).

Eine einfache Möglichkeit, Quellcode-Dateien in einer *.pdf-Datei darzustellen, bietet L^AT_EX: Sie finden im Digicampus sowohl eine Anleitung als auch eine passende Vorlage, die Sie verwenden können. Die Verwendung von L^AT_EX ist *nicht* verpflichtend, Sie können die notwendige *.pdf-Datei auch auf anderen Wegen erstellen.

- **Korrektur:** Wenn Sie sich bei Ihrer Abgabe an die obigen Punkte gehalten haben, erhalten Sie nach ca. einer Woche die Korrektur Ihrer Abgabe via Vips. Darin finden Sie neben der Bewertung Ihrer Lösung auch hilfreiche Hinweise und Tipps, was Sie in Zukunft beachten sollten. Lesen Sie sich Ihre Korrektur sorgfältig durch und beachten Sie die darin enthaltenen Hinweise für die Zukunft – und besonders für die Klausur.

Hinweise zum Notenbonus

Auf jedem bewerteten Übungsblatt können Sie bis zu vier Übungspunkte sammeln, die am Ende des Semesters automatisch in Bonuspunkte auf die Klausur umgewandelt werden.

- Insgesamt ist es möglich, bis zu 53 Übungspunkte zu sammeln:
 - $12 \cdot 4 = 48$ Übungspunkte durch die Bearbeitung der Übungsblätter,
 - bis zu 5 Übungspunkte für die erfolgreiche Teilnahme an einem größeren Programmier-Projekt in der Mitte des Semesters.
- Für jeweils 3 gesammelte Übungspunkte erhalten Sie 0.5 Bonuspunkte auf die Klausur, **sofern** Sie die Klausur ohne die Bonuspunkte bestanden haben. Sie können allerdings nicht mehr als 6 Bonuspunkte für die Klausur sammeln.
- 6 Bonuspunkte ($\hat{=}$ 36 Übungspunkte) entsprechen einer Notenstufe. Mit diesem („vollen“) Notenbonus können Sie Ihre Note also zum Beispiel von 2.3 auf 2.0 oder von 1.7 auf 1.3 verbessern. Eine Verbesserung von 4.3 auf 4.0 ist aber **nicht** möglich.
- Der Notenbonus zählt auch für die Wiederholungsklausur, nicht aber für Klausuren in späteren Semestern.

UML-Standard

Für die Modellierungssprache UML existieren verschiedene **Standards** (Spezifikationen), die im Laufe der Zeit zur Normierung der Sprache von speziellen Komitees entwickelt wurden bzw. immer noch werden.

Weil die Vorgaben dieser Standards im Bezug auf grafische Elemente manchmal vage sind, verwenden wir in der Veranstaltung *Informatik 2* einen eigenen Stil, den Sie im Laufe der Vorlesung kennen lernen – halten Sie sich also an den Stil der Vorlesung!

UML-Modellierungsumgebungen

Empfohlene Modellierungsumgebungen für die Erstellung von Klassenkarten und UML-Diagrammen:

- *UMLet*
<https://www.umlet.com/>
- *UMLetino*
<https://www.umletino.com/>

Java-Standard

Für die Programmiersprache Java existieren verschiedene **Standards** (Spezifikationen), die im Laufe der Zeit zur Normierung der Sprache von speziellen Komitees entwickelt wurden bzw. immer noch werden.

In der Veranstaltung *Informatik 2* werden wir uns an den Standard **Java SE 21** (häufig kurz: *Java 21*) halten, der die aktuelle LTS (Long Term Support) - Version ist.

- *OpenJDK 21* Bibliotheken, Compiler, Interpreter
<https://adoptium.net/> (Temurin)
- *Java SE 21 - API* (vgl. Standard-Bibliothek in C)
<https://docs.oracle.com/en/java/javase/21/docs/api/index.html>

Achtung:

- Achten Sie darauf, dass auf Ihrem System bzw. in Ihrer Entwicklungsumgebung die geforderte Variante des *JDK* (Java Development Kit) installiert ist.
- Achten Sie bei Suchanfragen in Suchmaschinen darauf, dass nicht auf eine veraltete API verwiesen wird (leicht zu erkennen daran, dass z.B. eine andere Zahl als die 21 im Link zur API steht).
- Die Verwendung von **unnamed classes**, **instance main methods** und des Schlüsselworts **var** ist in den Übungsblättern und in der Klausur **nicht erlaubt**.

Entwicklungsumgebung (IDE)

Die wichtigsten Entwicklungsumgebungen für die Entwicklung von Java-Programmen:

- *Eclipse* (Eclipse IDE for Java Developers)
<https://www.eclipse.org/>
- *IntelliJ IDEA*
<https://www.jetbrains.com/idea/>

Alle genannten IDEs sind empfehlenswert und besitzen unterschiedliche Vorteile. Es ist ausreichend, eine der genannten Umgebungen zu verwenden.

Trotzdem sollen Sie in der Lage sein, ein Java-Programm auch auf der Kommandozeile zu kompilieren und auszuführen.

Coding Conventions

Ein *guter Programmierstil* zeichnet sich dadurch aus, dass sich der Programmierer an vorgegebene Konventionen hält, die sich im Laufe der Zeit als vorteilhaft erwiesen haben. Wesentliche Gründe für die Einhaltung eines guten Stils sind:

- Bessere Lesbarkeit
- Bessere Wartbarkeit
- Bessere Team-/Projektarbeit
- Geringere Fehleranfälligkeit

Es gibt allerdings nicht *den richtigen Stil*, sondern lediglich eine Menge an verschiedenen Vorschlägen, die viele Gemeinsamkeiten aufweisen. Wichtig ist es deshalb, sich an firmen-/projekt-spezifische Vorgaben halten zu können, indem man die Grundlagen lernt.

Beispielhafte Style Guides für Java:

- *Java Coding Style Guide*
<https://user.eng.umd.edu/~austin/ence489c.d/java-style.html>
- *Google Java Style*
<https://google.github.io/styleguide/javaguide.html>

In dieser Veranstaltung werden wir uns an die Vorgaben des *Java Coding Style Guide* halten. Betrachten Sie den *Google Java Style* als optionale Ergänzung.

Achten Sie bei allen zukünftigen Übungsaufgaben auf **Kompilierbarkeit** und die **Einhaltung der Coding Conventions**. Kommentieren Sie gegebenenfalls nicht funktionierende Programmteile aus, um ein minimales kompilierbares Programm abgeben zu können. Schreiben Sie in den auskommentierten Teil auch, was dieser bezwecken sollte.

Als Zeichenkodierung verwenden Sie **UTF-8**.

Java Coding Style Guide – Ein kleiner Auszug

Die in diesem Auszug gegebenen Hinweise zum Stil sind ausschließlich Hinweise für Code-Elemente, die Sie in den ersten Wochen kennenlernen werden. Erweitern Sie Ihr Wissen durch Nachlesen in den oben genannten Dokumenten **selbstständig**, sobald neue Elemente in der Vorlesung vorgestellt werden!

- In Eclipse:
 - Window → Preferences → Java → Code Style → Formatter
Hier als Profil „Java Conventions“ auswählen.
 - Automatisches Formatieren: Source → Format (Shift + Ctrl + F)
- Automatisches Formatieren in IntelliJ: Code → Reformat File (Ctrl + Alt + L)

1. Sprechende Namen für Variablen/Methoden im *lowerCamelCase*:

FALSCH:	RICHTIG:
<code>int avh;</code>	<code>int alterHund;</code>
<code>int AlterHund;</code>	
<code>int alter_hund;</code>	

2. Ein Tab einrücken pro Verschachtelungstiefe (== 1x Tab-Taste pro {}-Block):

FALSCH:	RICHTIG:
<code>if (a == b) {</code>	<code>if (a == b) {</code>
<code> for (...) {</code>	<code> for (...) {</code>
<code> x = 4;</code>	<code> x = 4;</code>
<code> y = 3;</code>	<code> y = 3;</code>
<code> }</code>	<code> }</code>
<code> z = 6;</code>	<code> z = 6;</code>
<code>}</code>	<code> } else {</code>
<code>else { a = 8; }</code>	<code> a = 8;</code>
	<code>}</code>

3. Geschweifte Klammer immer in der Zeile des Methodenkopfes:

FALSCH:	RICHTIG:
<code>int meineMethode()</code>	<code>int meineMethode() {</code>
<code>{</code>	<code> ...</code>
<code> ...</code>	<code>}</code>
<code>}</code>	

4. Leerzeichen richtig setzen:

FALSCH:	RICHTIG:
<code>(a,b,c)</code>	<code>(a, b, c)</code>
<code>(a == b)</code>	<code>(a == b)</code>
<code>(a+b)</code>	<code>(a + b)</code>
<code>while(x != 0){</code>	<code>while (x != 0) {</code>

Hinweise zu den Programmieraufgaben:

Achten Sie bei allen zukünftigen Programmieraufgaben auf *Kompilierbarkeit* und *Einhaltung der Coding Conventions*; auch dann, wenn es nicht explizit im Aufgabentext gefordert ist.

Gehen Sie davon aus, dass alle Programme in der Programmiersprache Java zu erstellen sind.

Die Aufgaben setzen die Inhalte des Anpassungskurses für Java voraus. Wenn Ihnen die Bearbeitung der Aufgaben sehr schwer fällt, arbeiten Sie diese Inhalte nochmal durch.

Aufgabe 1 (Einfache Programme ohne Kommandozeilenparameter, 23 Minuten)

In den folgenden Teilaufgaben sollen Sie jeweils eine Programmklasse implementieren. Benennen Sie die Programmklasse jeweils nach der Nummer der Aufgabe.

Hinweis für alle Teilaufgaben: Mit Ausgabe ist die Ausgabe auf Kommandozeile (=Standardausgabe) gemeint.

- a) (*, 2 Minuten)
Schreiben Sie ein Java-Programm, das den Betrag von $\sin^{-1}(-0.5) = \arcsin(-0.5)$ berechnet und ausgibt. Verwenden Sie dazu die Methoden `abs` und `asin` der Klasse `Math`.
- b) (*, 3 Minuten)
Schreiben Sie ein Java-Programm, welches das Maximum von $\ln(10000)$ und e^3 bestimmt und ausgibt. Verwenden Sie dafür geeignete Klassenmethoden der Klasse `Math`.
- c) (**, 7 Minuten)
Schreiben Sie ein Java-Programm, das ...
- mithilfe einer geeigneten Klassenmethode der Klasse `Math` drei zufällige Zahlen größer oder gleich 0.0 sowie kleiner als 1.0 erzeugt und in Variablen speichert,
 - alle drei Werte jeweils in einer eigenen Zeile ausgibt,
 - mit mehreren `if`-Statements das Maximum der drei Zahlen bestimmt und ausgibt und
 - mit zwei Aufrufen einer geeigneten Klassenmethode der Klasse `Math` das Maximum der drei Zahlen bestimmt und ausgibt.
- d) (*, 3 Minuten)
Schreiben Sie ein Java-Programm, das 500-mal das Zeichen `j` ausgibt und alle 200 Zeichen einen Zeilenumbruch ausgibt.
- e) (*, 3 Minuten)
Schreiben Sie ein Java-Programm, das 1000-mal eine Zufallszahl aus dem Intervall $[0, 1[$ berechnet und am Ende die Anzahl der berechneten Zufallszahlen im Intervall $]0.4, 0.6[$ ausgibt.
- f) (**, 5 Minuten)
Schreiben Sie ein Java-Programm, das 100-mal einen zufälligen `boolean`-Wert bestimmt und beim Wert `true` eine ganzzahlige Zufallszahl aus dem Intervall $[1, 1000]$ berechnet und ihren quadrierten Wert ausgibt.
Hinweis: In Java können mit der Klasse `Random` aus dem Paket `java.util` verschiedene Objekte mit zufälligen Werten generiert werden, nicht nur Zufallszahlen aus dem Intervall $[0, 1[$ wie mit der Methode `Math.random()`.

Aufgabe 2 (Einfache Programme mit Kommandozeilenparameter, 21 Minuten)

In den folgenden Teilaufgaben sollen Sie jeweils eine Programmklasse implementieren. Benennen Sie die Programmklasse jeweils nach der Nummer der Aufgabe.

Hinweis für alle Teilaufgaben: Mit Ausgabe ist die Ausgabe auf Kommandozeile (= Standardausgabe) gemeint.

a) (**, 4 Minuten)

Schreiben Sie ein Java-Programm, das die Anzahl des Zeichens **e** im ersten Kommandozeilenparameter berechnet und ausgibt. Falls kein Kommandozeilenparameter übergeben wurde, soll **Keine Kommandozeilenparameter** ausgegeben werden.

Hinweis: Benutzen Sie eine geeignete Methode der Klasse **String** für den Zugriff auf einzelne Zeichen eines Strings.

b) (*, 3 Minuten)

Schreiben Sie ein Java-Programm, das die Längen der übergebenen Kommandozeilenparameter addiert und die berechnete Summe in Form eines ganzen Satzes ausgibt.

Beispiel: Es wurden insgesamt 25 Zeichen als Kommandozeilenparameter übergeben.

Hinweis: Benutzen Sie eine geeignete Methode der Klasse **String**, um die Länge eines Strings zu bestimmen.

c) (**, 6 Minuten)

Schreiben Sie ein Java-Programm, das genau einen Kommandozeilenparameter erwartet. Dieser soll nach jedem auftretenden Komma aufgeteilt und die entstehenden Teilzeichenketten jeweils in einer eigenen Zeile ausgegeben werden. Werden weniger oder mehr Kommandozeilenparameter übergeben, soll das Programm den String **Fehler** in die Standardfehlerausgabe **System.err** schreiben und dann abbrechen.

Beispiel: Die Verwendung der Strings "**eier,mehl,salz**" als Kommandozeilenparameter resultiert in der Ausgabe

eier

mehl

salz

Hinweis: Benutzen Sie dazu die **split**-Methode der Klasse **String**, um den String in mehrere Strings aufzuteilen.

d) (***, 8 Minuten)

Schreiben Sie ein Java-Programm, das alle übergebenen Kommandozeilenparameter aneinanderhängt, wenn mehr als ein Kommandozeilenparameter übergeben wurde, und alle Zeichen, die keine Ziffern sind, durch einen Unterstrich **_** ersetzt. Anschließend soll der resultierende String ausgegeben werden. Benutzen Sie dazu die **replaceAll**-Methode der Klasse **String** und recherchieren Sie selbstständig die Konstruktion eines geeigneten **regulären Ausdrucks** zur Übergabe an **replaceAll**. Falls weniger als 2 Kommandozeilenparameter übergeben werden, soll das Programm den String **Fehler** in die Standardfehlerausgabe **System.err** schreiben und dann abbrechen.

Hinweis: Benutzen Sie eine geeignete Methode der Klasse **String**, um einen String an einen anderen anzuhängen.

Aufgabe 3 (Einfache Methoden implementieren, 18 Minuten)

In dieser Aufgabe sollen Sie eine Programmklasse erstellen, die neben der `main`-Methode zwei weitere Methoden `removeAllVowels` und `appendTXT` enthält.

a) (****, Methode `removeAllVowels`, 8 Minuten*)

Implementieren Sie eine Klassenmethode `removeAllVowels`, die aus einer als Parameter übergebenen Zeichenkette alle Vokale entfernt und die resultierende Zeichenkette zurückgibt (den Methodenkopf legen Sie selbst fest).

Beispiel: Aus `Informatik` wird so `nfrmtk`.

Hinweis: Die Klasse `String` verfügt über verschiedene Methoden, um Zeichen zu ersetzen. Sie könnten jeden Vokal einzeln ersetzen, besser ist es aber, einen passenden **regulären Ausdruck** zu konstruieren und alle notwendigen Zeichen auf einmal zu ersetzen.

b) (*** , Methode `appendTXT`, 5 Minuten*)

Implementieren Sie eine Klassenmethode `appendTXT`, die an eine als Parameter übergebene Zeichenkette `TXT` anhängt, falls diese noch nicht damit endet, und die gegebenenfalls überarbeitete Zeichenkette zurückgibt (den Methodenkopf legen Sie selbst fest).

Hinweis: Zwar kann man die geforderten Änderungen auch selbst auf Zeichenebene implementieren, aber die Klasse `String` verfügt bereits über passende Methoden.

c) (*** , Methode `main`, 5 Minuten*)

In der `main`-Methode überprüfen Sie zuerst, ob mindestens ein Kommandozeilenparameter übergeben wurde. Falls nicht, soll das Programm lediglich die Zeichenkette `Fehler` in die Standardfehlerausgabe `System.err` schreiben. Ansonsten entfernen Sie zuerst mit `removeAllVowels` aus den übergebenen Kommandozeilenparametern alle Vokale und stellen mit `appendTXT` sicher, dass alle bearbeiteten Parameter mit `TXT` enden. Anschließend geben Sie jeden (bearbeiteten) Kommandozeilenparameter durch Kommata getrennt aus.

Aufgabe 4 (Vektoren, 22 Minuten)

In dieser Aufgabe sollen Sie eine Programmklasse zur Verwaltung von dreidimensionalen Vektoren erstellen, die neben der `main`-Methode vier weitere Methoden

- `myVectorRandom`,
- `myVectorPrint`,
- `myScalarProduct` und
- `myCrossProduct`

nach den Vorgaben der unteren Teilaufgaben enthält. In allen Aufgaben ist mit Ausgabe die Ausgabe auf Kommandozeile (=Standardausgabe) gemeint. Ein Vektor soll dabei durch ein eindimensionales `double`-Array repräsentiert werden.

Beispiel: `{0, -0.5, 7}` ist ein solcher dreidimensionaler Vektor.

a) (*** , die Methode `myVectorRandom`, 4 Minuten*)

Implementieren Sie eine Klassenmethode `myVectorRandom`, die einen dreidimensionalen Vektor mit zufälligen Einträgen vom Typ `double` erzeugt. Die zufälligen Werte generieren Sie mit der `random`-Methode der Klasse `Math`. Die Methode soll den erzeugten Vektor zurückgeben (den Methodenkopf legen Sie geeignet selbst fest).

b) (*** , die Methode `myVectorPrint`, 4 Minuten*)

Implementieren Sie eine Klassenmethode `myVectorPrint`, die einen übergebenen Vektor übersichtlich auf Kommandozeile ausgibt (den Methodenkopf legen Sie geeignet selbst fest).

c) (**, die Methode `myScalarProduct`, 4 Minuten)

Implementieren Sie eine Klassenmethode `myScalarProduct`, die das Skalarprodukt zweier Vektoren berechnet und zurückgibt. Beide Vektoren werden als Eingabeparameter übergeben (den Methodenkopf legen Sie geeignet selbst fest).

Das Skalarprodukt zweier dreidimensionaler Vektoren ist eine Zahl, die wie folgt berechnet wird:

$$\begin{pmatrix} u_1 \\ u_2 \\ u_3 \end{pmatrix} \circ \begin{pmatrix} v_1 \\ v_2 \\ v_3 \end{pmatrix} = u_1 \cdot v_1 + u_2 \cdot v_2 + u_3 \cdot v_3$$

d) (**, die Methode `myCrossProduct`, 6 Minuten)

Implementieren Sie eine Klassenmethode `myCrossProduct`, die das Kreuzprodukt zweier Vektoren berechnet und zurückgibt. Beide Vektoren werden als Eingabeparameter übergeben (den Methodenkopf legen Sie geeignet selbst fest).

Das Kreuzprodukt zweier dreidimensionaler Vektoren ist ein weiterer dreidimensionaler Vektor, dessen Einträge wie folgt berechnet werden:

$$\begin{pmatrix} u_1 \\ u_2 \\ u_3 \end{pmatrix} \times \begin{pmatrix} v_1 \\ v_2 \\ v_3 \end{pmatrix} = \begin{pmatrix} u_2 \cdot v_3 - u_3 \cdot v_2 \\ u_3 \cdot v_1 - u_1 \cdot v_3 \\ u_1 \cdot v_2 - u_2 \cdot v_1 \end{pmatrix}$$

e) (**, die Methode `main`, 4 Minuten)

In der `main`-Methode

- erzeugen Sie zuerst zwei dreidimensionale Vektoren u und v mit zufälligen **double**-Einträgen,
- geben diese aus,
- berechnen das Skalarprodukt beider Vektoren und geben es aus und
- berechnen das Kreuzprodukt $u \times v$ beider Vektoren und geben es aus.